단국대학교 소프트웨어학과 졸업작품 결과보고서

EDRdog: 행위 기반 엔드포인트 위협 탐지·대응 시스템

엔드포인트 행위 로그로 공격을 탐지하고, 사람이 확인한 뒤 프로세스를 종료하는 EDR

학과	소프트웨어학과
성명	김동현
학번	32210492

단국대학교

목차

1	개요	3
	배경, 목적, 범위와 대상 환경	
2	요구사항	4
	사용자, 기능 요구사항, 비기능 요구사항, 요구사항 ID 체계	
3	시스템 설계	6
	아키텍처, 구성 요소별 역할, 데이터 흐름, 기술 스택	
4	구현	9
	에이전트, 수집, 탐지, 대응, 알림, 조회 API, 웹 콘솔 화면	
5	주요 기술 결정	17
6	문제 해결 사례	18
7	테스트와 검증	19
	자동 테스트, 플랫폼 검증, KISA 기준 시험, 성능 측정	
8	배포와 운영	22
9	결론, 한계와 향후 과제	24
부록	저장소와 실행 방법	26

그림 목록

그림 1. EDRdog 시스템 아키텍처

그림 2. 서비스 간 호출과 저장소 구성

그림 3. 프로세스 종료(kill) 흐름

그림 4. 대시보드 화면

그림 5. 위협 보드 화면

그림 6. 사건 상세 화면

그림 7. 엔드포인트 상세 화면

그림 8. 수집 로그 화면

그림 9. 수집 알림 연동 화면

1 개요

1.1 배경: 백신과 EDR의 차이

EDR(Endpoint Detection and Response)은 개별 기기에서 벌어지는 일을 관찰해 위협을 찾고, 대응까지 하는 보안 체계이다. 백신은 파일이 무엇인지를 본다. 알려진 악성코드의 지문과 대조한다. EDR은 파일이 무엇을 했는지를 본다. 무엇을 실행했는지, 어디에 접속했는지, 어떤 순서로 일어났는지를 본다.

지문이 없는 새 공격은 백신을 통과한다. 개별로는 정상인 행동의 조합은 EDR만 잡을 수 있다. 예를 들어 파일 다운로드와 파일 실행은 각각 정상이지만, 방금 받아온 그 파일을 곧바로 실행하는 것은 의심스럽다.

구분	백신	EDR
보는 것	파일 자체(알려진 악성코드 지문 대조)	파일의 행위(실행, 접속, 순서)
판단 질문	이 파일이 악성코드 목록에 있는가	이 파일이 방금 외부에서 받아온 것이고, 받자마자 실행되었는가
지문 없는 새 공격	통과한다	행위 순서로 탐지할 수 있다
개별로는 정상인 행동의 조합	잡지 못한다	잡을 수 있다

1.2 목적

> 엔드포인트에서 수집한 행위 로그로 공격을 탐지하고, 그 기기의 프로세스를 실제로 종료하는 EDR을 만든다.

수집, 판정, 대응(Slack 알림과 프로세스 종료 명령)까지 이어지는 파이프라인을 한 사람이 끝까지 구현하는 것이 목표이다. 대응은 사람이 화면에서 확인한 뒤에만 실행되도록 설계했다.

1.3 범위와 대상 환경

대상 환경	에이전트는 macOS와 Windows를 대상으로 한다. Linux 등 그 외 플랫폼은 지원하지 않는다.
포함 범위	이벤트 수집, 공격 판정, Slack 알림, 프로세스 종료 명령, 조회 API, 웹 콘솔, 배포와 운영
대응 범위	프로세스 종료 하나로 한정한다. 자동 실행은 없고, 사용자가 확인 단계를 거쳐야 실행된다.
조직 단위	데이터와 대응은 조직(tenant) 단위로 분리한다.
개발 기간	커밋 기준으로 Backend 2026-07-19 ~ 2026-11-11, Frontend 2026-07-22 ~ 2026-11-11
저장소와 라이선스	Backend, Frontend 두 저장소로 나뉜다. 에이전트(agent/)만 GPL-3.0이다(ETW 라이브러리가 GPL-3.0). 나머지는 MIT이다.
기준 시점	이 보고서는 2026-11-11 시점의 원격 main(Backend 92ff3e6, Frontend e7a320b)을 기준으로 한다.

시험 기준. 완성 단계에서 KISA EDR 성능평가 항목(V1.4, 2026-05-08 개정)에 맞춰 탐지·대응 시험과 보안 기능 시험을 2026-11-10~11에 자체 수행했다. 공식 평가가 아니다. 결과와 조건은 7장에 적었다.

2 요구사항

2.1 대상 사용자

사용자	할 수 있는 일
관리자(admin)	설치 링크 발급, 알림 판단, 프로세스 종료 지시, 보안 설정, 감사기록과 자체시험 조회
조회 전용(viewer)	같은 조직 안에서 조회만 한다. 판단과 대응 버튼, PLATFORM 메뉴는 숨기고 서버도 403으로 막는다. admin이 viewer 계정을 만든다.
처음 방문자	로그인 없이 데모 데이터로 서비스를 체험한다.
공개 데모 계정	관리 IP, 보안정책, 계정 생성을 바꿀 수 없다(403).

호스트 소유자는 개인 Slack webhook으로 알림을 받을 수 있다.

2.2 기능 요구사항

ID	기능	내용
F1	**행위 수집**	macOS와 Windows에서 process, network, file, script, dns, I7 여섯 종류의 이벤트를 수집한다.
F2	**등록과 조직 구분**	조직별 enroll secret으로 등록하고 이후 X-Node-Key로 인증한다. 서버 인증서를 고정(pinning)한다. 대시보드가 만든 설치 링크(기본 TTL 24시간) 한 줄로 설치한다.
F3	**공격 판정**	코드 규칙(R1~R5)과 SigmaHQ 기반 데이터 규칙으로 판정한다. 시퀀스는 host별 5분 윈도우로 엮는다. MITRE ATT&CK에 매핑한다.
F4	**알림**	Slack Incoming Webhook으로 보낸다. 개인 webhook이 우선이고, 없으면 조직 webhook, 둘 다 없으면 보내지 않는다. 같은 알림은 60초 동안 억제한다.
F5	**대응**	kill_process. 대시보드에서 사람이 확인해야 실행된다.
F6	**저장과 조회**	이벤트 7일, 판정 90일, 시간별 롤업 180일 보존한다.
F7	**대시보드 API**	인증, 알림, 이벤트, 호스트, 인시던트, 통합 검색, 인텔리전스, 운영 상태, 조직 설정, 관리 IP, viewer 계정 생성, 감사기록 조회, 서버 자체시험
F9	**보안 기능 관리**	admin 전용. 패스워드 규칙, 연속 실패 잠금, 보안정책, 기본 패스워드 강제 변경, 관리 IP 허용 목록, 감사기록, 서버와 에이전트의 자체시험과 무결성 검증

명세 번호 F8(공통 기능: 통합 검색, 자동 새로고침, 테마, 반응형)은 이 표에서 생략했다.

2.3 비기능 요구사항

분류	요구	구현
조직 분리	다른 조직의 데이터와 대응이 섞이지 않는다.	서버가 node_key로 tenant를 결정한다. 모든 조회 질의에 TenantScope로 tenant 조건을 강제한다.
통신 보안	에이전트 통신을 보호한다.	HTTPS, 서버 인증서 고정, node_key는 SHA-256 해시로만 저장한다.

분류	요구	구현
대응 안전	임의 코드 실행 경로를 만들지 않는다.	명령 종류를 kill_process 하나로 고정한다. 종료 대상은 요청 본문이 아니라 알림에서 꺼낸다.
수집 신뢰성	수집한 이벤트를 잃지 않는다.	collector가 브로커 ack를 기다리고 실패하면 503을 돌려준다. 에이전트는 등록 실패 시에도 간격을 늘려 재시도한다.
탐지 지속성	규칙 문제로 탐지가 통째로 꺼지지 않는다.	깨진 규칙 파일은 이전 규칙을 유지한다. 한 규칙의 예외가 나머지 판정을 막지 않는다. Streams 상태를 health에 반영한다.
운영 편의	규칙을 재배포 없이 바꾼다.	규칙 디렉터리를 30초마다 다시 읽는다. 운영은 ConfigMap으로 규칙만 교체한다.
보존	용도별 보존 기간을 둔다.	이벤트 7일, 판정 90일, 시간별 롤업 180일. S3 Parquet 아카이브는 기본 OFF이다.
사용성	처음 방문자가 서비스를 볼 수 있다.	로그인 없이 데모 데이터로 화면을 체험한다.

2.4 요구사항 ID 체계

백엔드는 docs/requirements 문서에 영역별 기준 ID를 두고, 각 ID에 검증 방법과 테스트 메서드를 적었다. 번호가 붙은 기준은 거의 전부 자동 테스트로 확인하고, 실기기가 필요한 항목은 별도 M 번호(예: AGT-M1)로 수동 확인한다. 프론트엔드는 화면 영역별 ID를 두었으며 대부분 수동 확인이다.

저장소	영역	접두사와 요구사항 ID 수	대표 기준
Backend	**수집**	`AGT 44`	본문 8MB 상한 413, 깨진 gzip 400, 브로커 ack 실패 시 503, 틀린 node_key 거부
Backend	**탐지**	`DET 27, SUP 4, TEN 2, ING 4` (요구사항 ID 수)	5분 윈도우 밖 조합은 알림 아님, 다른 host 이벤트 상관 안 함, 룰 파일 변경 시 재배포 없이 반영, 테넌트 조건 강제
Backend	**알림**	`ALR 21`	수신 대상 결정, webhook은 https만, 발송 실패 시 쿨다운 되돌림
Backend	**대응**	`RSP 38`	kill 요청은 PENDING으로 즉시 반환, 같은 명령 한 번만 실행, 다른 tenant 대상 명령은 내려가지 않음
Backend	**조회**	`QRY 27`	모든 질의에 tenant 조건, limit과 offset 상한 초과 시 400
Backend	**보안**	`SEC 90`	감사기록, 자체시험과 무결성, 인증과 세션, 관리 IP와 접근통제, KISA 평가 후속 수정
Frontend	**공통, 조사**	`SH 19, INV 18`	비로그인 데모, 401 처리, 서버 페이지네이션, 서버 검색
Frontend	**분석, 대응**	`ANA 13, RES 18`	관계 그래프 높이 제한, 폴링 40초 상한, 자동 실행 없음
Frontend	**보안, 랜딩**	`KS 21, LD 15`	로그인 실패 한 문장, 비밀번호 규칙 실시간 표시, viewer 숨김, 링크 공유 미리보기

프론트엔드의 자동 확인 기준은 SH-2(echarts 화면이 lazy 청크)와 SH-19(아이콘 라이브러리 금지) 둘이며 `scripts/check.sh`가 검사한다. 나머지는 수동 확인이다.

3 시스템 설계

3.1 전체 아키텍처

그림 1은 시스템 전체 구조이다. 수집 영역에서 Go 에이전트가 이벤트를 HTTPS로 Collector Server에 보내면, Collector는 인증과 검증을 거친 이벤트를 Kafka `events` 토픽에 발행한다. 데이터 파이프라인 영역에서는 Kafka Streams(그룹 detector)와 ClickHouse 적재(그룹 archiver)가 이 토픽을 각자의 속도로 소비한다. 판정은 `alerts` 토픽으로 나가 Responder Service와 Alert Service(Slack)가 소비하고, API Server는 ClickHouse를 읽어 웹 콘솔(React)에 응답한다.

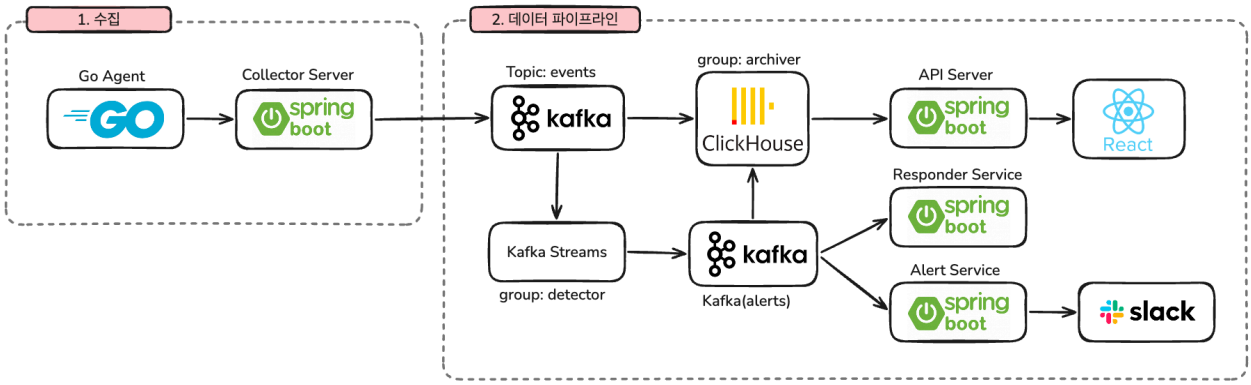


그림 1. EDRdog 시스템 아키텍처. 그림의 Kafka Streams(group: detector)는 detector-service, ClickHouse 적재(group: archiver)는 archiver-service에 해당하고, Collector Server, Responder Service, Alert Service, API Server는 각각 같은 이름의 서비스이다.

3.2 구성 요소별 역할

구성	역할
agent (Go)	커널 이벤트 구독, 버퍼링, 서버 전송, 조치 실행, 자체시험과 감사기록 전송. 단일 실행 파일, macOS와 Windows
collector-service	에이전트의 HTTP 입구. 인증, tenant 태깅, 검증 후 통과분만 `events`에 발행한다. 에이전트 감사기록을 api-service로 중계한다.
detector-service	Kafka Streams로 host별 상관분석(시퀀스 규칙)과 단일 이벤트 규칙 판정을 하고 `alerts`에 발행한다.
alert-service	`alerts`를 소비해 tenant별 Slack webhook으로 발송한다.
responder-service	`alerts`를 소비해 dry-run 권고를 만들고, kill 명령 큐를 MySQL에 보관한다.
archiver-service	`events`와 `alerts`를 소비해 ClickHouse에 적재하고 S3로 내보낸다.
api-service	프론트용 읽기 API, 인증, tenant와 알림 설정, 설치 링크, 접근통제, 감사기록, 서버 자체시험
event-schema	`events` 토픽의 .proto와 Kafka, 관측 유틸. 서비스가 아니라 라이브러리이다.
웹 콘솔 (Frontend)	알림, 이벤트, 호스트, 사건을 보여주고 확인을 거쳐 프로세스 종료를 지시한다. api-service만 호출한다.

3.3 데이터 흐름

1. Go 에이전트가 HTTPS로 collector에 enroll, heartbeat, events, command-result, audit을 호출한다.

2. collector가 인증, tenant 태깅, 검증 후 `events` 토픽(3파티션, 키 host, Protobuf)에 발행한다.
3. detector(컨슈머 그룹 detector)와 archiver(그룹 archiver)가 `events`를 각자 속도로 소비한다. detector는 `alerts` 토픽(3파티션, JSON)에 판정을 발행한다.
4. alert-service는 Slack으로 알리고, responder는 권고를 만들고, archiver는 ClickHouse에 적재한다. 원본은 ClickHouse에서 S3 Parquet으로 내보낼 수 있다.
5. 웹 콘솔은 api-service만 호출한다. api-service가 ClickHouse(읽기)와 MySQL(상태)을 합쳐 응답한다.

responder로 가는 직접 호출 두 경로는 `X-Internal-Key` 헤더로 인증한다. responder의 `/api/responder/**`는 InternalKeyFilter 한 곳에서만 연다.

그림 2는 그림 1에 없는 MySQL과 서비스 간 직접 호출, kill 명령 경로를 더해 같은 구조를 다시 그린 것이다.

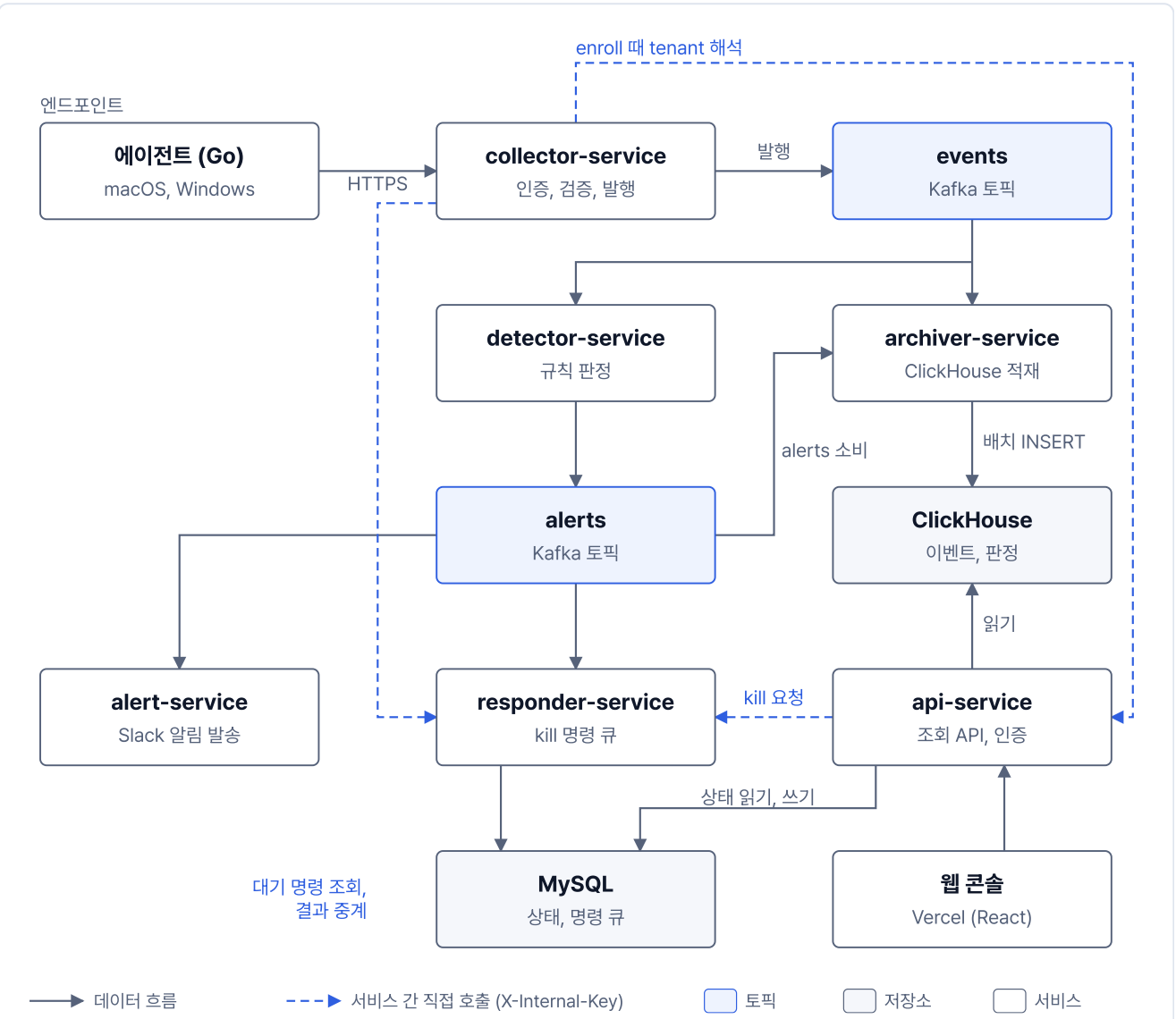


그림 2. 서비스 간 호출과 저장소 구성. 그림 1에 없는 MySQL(상태, 명령 큐)과 서비스 간 직접 호출, kill 명령 경로를 더해 같은 구조를 다시 그렸다. 서비스 간 직접 호출은 토픽을 거치지 않는 예외이다. 그림에는 세 곳(collector에서 api-service의 enroll 때 tenant 해석, collector에서 responder의 대기 명령 조회와 결과 중계, api-service에서 responder의 kill 요청)을 점선으로 그렸고, collector에서 api-service(audit 중계), api-service에서 collector(등록 노드 조회), alert-service에서 api-service(webhook과 알림 대상 조회)는 생략했다.

3.4 설계 원칙

- detector는 저장소를 쓰지 않는다. 상태는 Kafka Streams 상태 저장소에만 둔다. ClickHouse 쓰기는 archiver만 한다.
- responder는 내부 키로만 열고 외부에 직접 노출하지 않는다. 서비스 사이는 토픽을 거치는 것이 기본이다.
- 관리 API 접근통제는 api-service의 AccessInterceptor 한 관문(로그인, 조직 허용 IP, 역할)에서 본다. 컨트롤러는 `@RequiresRole`만 붙인다.
- Redis는 쓰지 않는다. 알림 쿨다운은 파티셔닝 덕에 인스턴스 로컬 메모리로 충분하다.

3.5 기술 스택

영역	기술
에이전트	Go. Windows는 ETW(golang-etw 사본을 third_party에 두고 패치), macOS는 eslogger(EndpointSecurity)와 libproc. DNS와 TLS는 패킷 캡처(Windows는 pktmon 기반)
서버	Java/Spring 서비스 6개. 외부 연동은 Slack Incoming Webhook, MaxMind GeoLite2(국가 코드)
메시징, 판정	Kafka(`events`, `alerts` 각 3파티션, 복제 1, 브로커 1대, 보존 기본 7일), Kafka Streams(RocksDB 상태 저장소와 changelog), Protobuf(events 값)
저장	ClickHouse(이벤트와 판정), MySQL(계정, 조직, 명령 큐, 알림 처리 상태, 감사기록, 자체시험 이력), S3/MinIO(Parquet 아카이브, 기본 OFF)
규칙 파일	Sigma 부분집합 YAML. 번들은 detector jar의 `resources/rules/`, 운영은 ConfigMap `detector-rules`
인프라	Kubernetes(로컬 kind, 운영 k3s), GHCR 이미지, GitHub Actions CD, Caddy, Infisical(시크릿), 관측은 로컬 otel-lgtm과 운영 New Relic
프론트엔드	Vite 8, React 19, TypeScript 6, Tailwind CSS v4, React Router 7, Zustand 5, echarts 6, world-atlas와 topojson-client, gsap(랜딩 전용), oxlint, Prettier. Vercel 배포

4 구현

4.1 에이전트와 수집

에이전트는 Go 단일 실행 파일이다. 공격 도구는 짧게 실행되고 사라지므로 주기 조회는 놓친다. 그래서 가능한 곳은 이벤트 구독 방식으로 수집한다.

플랫폼	이벤트	수집 방식
Windows	process	ETW Kernel-Process(ProcessStart)와 시스템 프로세스 프로바이더(명령행 보강)
	network, file	ETW Kernel-Network(TCP 연결 시도), Kernel-File(파일 생성)
	dns, l7	pktmon 기반 패킷 캡처(TLS SNI와 DNS)
macOS	process, file	eslogger의 exec, create, rename, unlink
	network	libproc 소켓 스냅샷 폴링(유일한 폴링)
	dns, l7	패킷 캡처

- 실행 파일이 셸이나 스크립트 인터프리터이면 process가 아니라 script 이벤트로 내보낸다. detector는 script에만 T1059 규칙을 건다.
- 서버와의 계약은 enroll, heartbeat(설정과 명령을 한 응답으로), events(최대 500건 배치, 기본 5초 주기), command-result, audit이다. 실패는 HTTP 상태 코드로 알리고 200 본문에 담지 않는다.
- 401을 받으면 에이전트가 스스로 재등록한 뒤 한 번 재시도한다. 센서가 전부 멈추면 에이전트는 종료 코드 1로 끝나고, Windows에서는 서비스 관리자가 다시 띄운다.
- 서버는 tenantId와 host를 에이전트가 보낸 값이 아니라 node_key로 푼 값으로 붙인다.
- 검증용으로 `-selftest`(커널 없이 가짜 이벤트를 보내 서버까지 경로 확인), `-security-check`(자체시험), `-record-baseline`(무결성 기준값 기록)을 둔다.

> **osquery와 Zeek를 버린 이유.** 처음 구조는 osquery와 Zeek 이원 구조였다. osquery는 소켓 주인을 모르고 Zeek는 PID를 모른다. 탐지 규칙 R2가 두 관점을 이어야 하고 Zeek는 Windows를 지원하지 않아, 단일 Go 에이전트로 합쳤다(ADR 008).

collector와 archiver

- collector는 본문 8MB 상한을 압축 여부와 무관하게 적용한다(초과 413, 깨진 gzip 400). 검증에 실패한 한 건이 배치 전체를 실패시키지 않는다.
- `events` 발행 본문은 Event 그대로이고, 레코드 헤더에 스키마 버전, 이벤트 타입, tenant를 싣는다. 파티션 키는 host이다.
- 브로커 ack를 요청 단위로 기다리고(상한 5초) 실패하면 503을 돌려준다. 이전에는 ack 실패에도 200을 주어 에이전트가 배치를 지워 유실이 생겼다(ADR 018).
- 압축은 구간별로 쓴다. 에이전트 구간은 gzip, Kafka 구간은 zstd이며 1KB 미만은 압축하지 않는다(ADR 013).
- archiver는 ClickHouse에 배치로 INSERT한다. events는 적재일로 파티션을 나누고, 시간별 롤업 테이블 `events_hourly`를 둔다.

4.2 탐지

핵심 개념은 단일 이벤트는 정상이고 여러 행동의 순서가 위협이라는 것이다. 판정은 두 갈래이다.

1. **코드 규칙 R1~R5**(`Rules.java`). 시퀀스(여러 이벤트를 시간 순으로 엮는 판정)는 R1과 R2뿐이고, R3~R5는 이벤트 하나로 판정한다.
2. **데이터 규칙**. SigmaHQ 공개 규칙을 이 에이전트가 보내는 필드에 맞게 옮긴 YAML 파일이다. 이벤트 하나로 판정되는 나머지 규칙은 모두 파일이다.

코드 규칙

규칙	잡는 대상	MITRE
`R1 SUSPICIOUS_PROCESS_CHAIN`	오피스 문서가 shell을 띄움(부모 office, 자식 shell). 심각도 HIGH, kill 권고	`T1059`
`R2 DOWNLOAD_AND_EXECUTE`	외부에서 받아온 파일을 실행(network 이벤트 뒤에 임시 또는 다운로드 경로 실행). CRITICAL, kill 권고	`T1105, T1204`
`R3 SCRIPT_FROM_TEMP_PATH`	임시나 다운로드 경로에서 스크립트 실행. 실행 대상만 본다(ADR 027)	`T1059`
`R4 FILE_IN_AUTORUN_PATH`	자동실행 경로에 파일 생성	`T1547`
`R5 WEAK_TLS_HANDSHAKE`	낡은 TLS(SSL 3.0, TLS 1.0, 1.1)로 맺은 연결	`T1573`

R2가 심각도가 가장 높다. 프로세스 관점(실행)과 네트워크 관점(다운로드 연결)을 이어야 성립하며, 수집기를 하나로 합친 이유가 바로 이 규칙이다. 단, R1은 실제 에이전트 이벤트에서 걸리지 않을 수 있다. 자식 shell이 script 타입으로 오는데 R1은 process 타입을 가정하며, 단위 테스트는 process로 만들어 통과했고 실데이터로는 확인하지 못했다.

시퀀스 판정

detector는 host별로 최근 이벤트를 상관 버퍼(Kafka Streams 상태 저장소)에 들고 있다가 새 이벤트가 올 때마다 대조한다. 윈도우는 5분이다. Flink나 Spark 같은 별도 클러스터 대신 Kafka Streams Processor API로 직접 작성했다. 상태를 들고 판정해야 하는데 DSL로는 버퍼 판정을 표현할 수 없고, 라이브러리라서 인스턴스를 늘리면 파티션이 자동으로 재분배되기 때문이다(ADR 002).

- 다른 host의 이벤트끼리는 상관하지 않는다. 같은 시퀀스의 알림은 한 번만 낸다.
- 조용해진 host의 대기 중 판정은 punctuator로 flush한다.
- 재시작해도 상관 버퍼는 상태 저장소와 changelog로 복원된다.

발생 시각 기준 정렬

판정은 도착 순서가 아니라 엔드포인트에서 찍힌 발생 시각 기준이다. 한 기기의 이벤트는 같은 파티션(키 host)에 들어가고, 버퍼는 새 이벤트보다 앞선 이벤트와 뒤선 이벤트를 양쪽으로 대조한다. 실제로 실행 이벤트가 네트워크 관측보다 먼저 도착하면 R2가 성립하지 않는 문제가 있었고, 이 방식으로 고쳤다(ADR 006). 워터마크는 host별로 둔다. 전역 stream-time을 쓰면 다른 단말의 이벤트가 모두 늦은 이벤트가 되기 때문이다.

데이터 규칙과 규칙 리로드

단일 이벤트 규칙은 Sigma 부분집합 YAML 파일 하나가 규칙 하나이다. 규칙을 하나 추가하는 데 코드 수정과 재배포가 필요했고 공개 SigmaHQ 규칙을 가져올 길이 없어서 도입했다(ADR 020).

항목	내용
name, tags	name을 대문자로 바꾼 값이 알림 ruleId이다. tags의 첫 `attack.tNNNN`이 MITRE 기법이다.
level	심각도를 정한다. informational, low, medium은 MEDIUM, high는 HIGH, critical은 CRITICAL이다. kill 권고는 명백히 악성인 번들 규칙에만 붙고 나머지는 알림만 보낸다.

항목	내용
logsource	process_creation, file_event, dns_query, network_connection
detection	modifier는 contains, startswith, endswith, all. condition은 and, or, not, 괄호, 1 of, all of. 지원하지 않는 구문은 그 규칙만 거절하고 로그에 이유를 남긴다.
Image 필드	에이전트가 파일명만 보내므로 파일명 일치로 읽는다. 에이전트가 보내지 않는 필드(레지스트리, 프로세스 접근 등)는 쓸 수 없다.
번들 종류	Windows 명령행(LOLBin 다운로드, 자격증명 덤프, 랜섬웨어 준비, 방어 무력화), macOS 명령행(권한 상승, 지속성, 방어 무력화), DNS(Tor, 파일 공유), 파일(mimikatz, lsass 덤프 흔적)

- 규칙 디렉터리(`DETECTION_RULES_DIR`)를 30초마다 다시 읽는다. 운영은 ConfigMap `detector-rules`를 `/etc/edrdog/rules`에 마운트해 재배포 없이 규칙만 바꾼다. ConfigMap은 `scripts/gen-rules-configmap.sh`가 규칙 파일에서 생성하며 직접 고치지 않는다.
- 볼륨은 optional이라 ConfigMap이 없어도 파드가 뜨고 이미지에 든 번들 규칙으로 동작한다.
- 깨진 파일은 그 파일의 이전 규칙을 유지한다. 디렉터리를 읽지 못하거나 비어 있어도 이전 규칙을 통째로 유지한다. 규칙 판정 중 예외는 그 규칙만 건너뛴다. 탐지가 통째로 꺼지는 경로가 없다.
- ConfigMap 마운트처럼 심볼릭 링크(`..data`)로 놓인 파일도 읽는다. 같은 문제는 상태가 바뀔 때만 ERROR로 남겨 로그가 쌓이지 않게 했다.
- 데이터 규칙은 R2(CRITICAL)를 가리지 않고, 심각도가 같으면 코드 규칙이 이긴다. 데이터 규칙은 시퀀스 트리거와 상관 버퍼 대상을 바꾸지 않는다.
- 운영 환경에서 ConfigMap 수정이 실제로 반영되는지는 수동 확인 항목(DET-M3)이다.

중복 알림 억제와 오탐 개선

- 판정은 at-least-once이다. EOS를 쓰지 않고 알림 단계에서 60초 억제한다(키는 수신 대상, host, ruleId). 문제는 중복 판정이 아니라 중복 알림이고, EOS는 처리량을 낮추고 지연을 늘리기 때문이다(ADR 003).
- R3가 정상 개발 작업(python venv, pip 설치)에 네 번 걸려, 실행 대상(첫 토큰, 인터프리터가 읽는 스크립트)만 보도록 바꿨다(ADR 027). kill을 권고하는 번들 규칙의 오탐도 줄이고 명령행 끝 변형을 잡도록 고쳤다.

4.3 대응: 프로세스 종료 흐름

대응 수단은 프로세스 종료(`kill_process`) 하나이다. 사용자가 확인하기 전에는 실행되지 않는다.

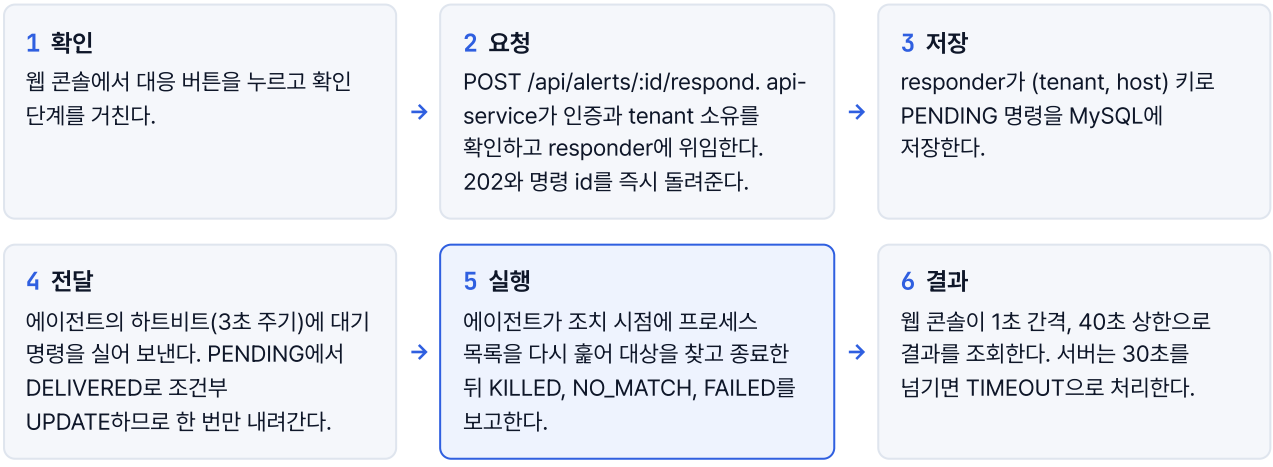


그림 3. 프로세스 종료(kill) 흐름. 서버는 기기에 접속하지 않고 하트비트 응답에 명령을 실어 보내므로 대상 기기에 포트를 열 필요가 없다.

안전장치

- **종료 대상은 알림에서 꺼낸다.** api-service는 tenant, host, target을 모두 알림에서 꺼내 responder에 싣는다. 요청 본문의 target은 받기만 하고 쓰지 않는다. 본문을 믿으면 로그인한 사용자가 그 기기의 아무 프로세스나 종료할 수 있기 때문이다.
- **명령 종류를 고정한다.** 셸 스크립트를 내려보내지 않고 서버는 대상 문자열만 전달한다. 스크립트 전송은 모든 엔드포인트에 임의 코드 실행 채널을 여는 일이고 플랫폼별 스크립트를 이중으로 관리해야 한다(ADR 007).
- **정확한 일치만 종료한다.** 경로 구분자가 있으면 전체 경로가 일치해야 하고, 없으면 파일명이 일치해야 한다. 부분 일치는 없다.
- **자기 자신과 PID 1은 종료하지 않는다.** 일부만 종료되면 FAILED로 보고한다.
- **중복 실행을 막는다.** 같은 명령을 다시 받으면 재실행하지 않고 저장된 결과만 다시 보고한다(최근 256건 기억). 모르는 명령 종류는 실행하지 않고 실패로 보고한다.
- **조직을 넘지 않는다.** 다른 tenant의 같은 이름 host에는 명령이 내려가지 않는다. 서버는 보고한 노드의 (tenant, host)가 명령 대상과 같을 때만 결과를 반영한다. 시한이 지난 명령은 내려가지 않는다.
- **권한을 나눈다.** responder는 X-Internal-Key 필터로만 열린다. viewer는 대응 API가 403이다.

상태	붙이는 쪽과 의미
`KILLED, NO_MATCH, FAILED`	에이전트가 보고한다. 종료 성공, 일치하는 프로세스 없음, 실패 또는 일부만 종료
`TIMEOUT, COOLDOWN, DISABLED`	서버가 붙인다. 시한 초과, 쿨다운 중, 비활성

실제 프로세스 종료는 2026-11-10 탐지·대응 자체 시험에서 확인했다. 알림 확인 후 kill 요청부터 KILLED 확인까지 4초(3초 간격 조회)였다. 대상은 임시 서명한 /bin/sleep 사본이며, 서명하지 않은 사본은 macOS가 실행 직후 종료시켜 본 시험에서는 NO_MATCH였고 재시험에서 KILLED였다(7.3절).

4.4 알림

- detector가 `alerts`에 발행한 판정을 alert-service가 소비해 Slack Incoming Webhook으로 보낸다.
- 수신 대상은 개인 webhook이 우선이고, 없으면 조직 webhook이며, 둘 다 없으면 보내지 않는다. webhook은 https만 허용한다.
- 같은 알림은 60초 동안 억제한다(키는 수신 대상, host, ruleId). 발송에 실패하면 쿨다운을 되돌려 다음 시도가 막히지 않게 한다.
- 한 건의 실패로 컨슈머가 멈추지 않는다. webhook 조회는 api-service를 호출하고 60초 캐시한다.

4.5 조회 API

api-service가 프론트용 읽기 API를 맡는다. ClickHouse(이벤트와 판정, 읽기)와 MySQL(처리 상태)을 합쳐 응답하며, 알림 위에 MySQL의 처리 상태를 겹쳐 보여준다.

그룹	내용
인증, 계정	가입, 로그인, 로그아웃, 내 정보, 패스워드 변경, 보안정책
탐지 조회	알림, 이벤트, 호스트, 인시던트(알림을 프로세스 계보로 묶은 사건), 통합 검색, 인텔리전스(egress 토폴로지, IP와 도메인 조회)
대응	알림에 대한 respond와 결과 조회
설정	조직 설정, 설치 링크, Slack webhook, 관리 IP, viewer 계정 생성

그룹	내용
운영, 보안	운영 상태, 감사기록 조회, 서버 자체시험과 무결성 검증

- 모든 질의에 tenant 조건을 강제한다(TenantScope). limit과 offset에 상한을 두고, 넘으면 400을 돌려준다.
- 접근통제는 AccessInterceptor 한 곳에서 로그인, 조직 허용 IP, 역할을 확인한다. 보안 기능 화면은 admin만 쓸 수 있다.
- 세션 토큰은 SHA-256으로 저장하고, 유휴 세션은 종료하며, 같은 계정의 새 로그인은 옛 세션을 종료한다(ADR 032). 패스워드 규칙, 연속 실패 잠금, 실패 사유 비공개를 적용한다(ADR 031).
- 감사기록은 MySQL `audit_log` 한 테이블에 쌓고 수정과 삭제 경로를 두지 않았다(ADR 028).

4.6 웹 콘솔

웹 콘솔은 Vite 8, React 19, TypeScript 6, Tailwind CSS v4로 만들었다. 페이지는 주로 hooks(useApi, usePagedList)를 거쳐 api(client, index, demo)로 백엔드 /api를 부른다. 사용자는 로그인하지 않고도 데모 데이터로 화면을 볼 수 있다.

화면	내용
/dashboard	가장 급한 미판단 위협 하나(세로 공격 흐름, 권고, 판정 버튼), 대기열, 엔드포인트 상태, 접히는 한눈에 보기(탐지 추이, 위협 TOP)
/threats	상태별 보드(미판단, 확정, 오탐). 열마다 따로 조회한다.
/endpoints	호스트 위험도, 프로세스 트리, 호스트의 위협, 사건, 로그
/events	수집 로그. 원시 이벤트를 원본 JSON까지 본다.
/incidents	알림을 프로세스 계보로 묶은 사건 카드, 증거 사슬, 타임라인
/intelligence, /lookup, /map	관계 분석(egress 토폴로지 그래프), IP와 도메인 조회(실시간 DNS), 위협 지도
/onboarding	수집 알림 연동. 에이전트 설치 링크, Slack webhook, 내 기기 등록
/settings, /audit, /selftest	비밀번호, 보안정책, 관리 IP, 계정, 감사기록, 자체점검. 서버는 admin만 허용한다.

메뉴는 사이드바 기준으로 비로그인 9개, viewer 8개, admin 12개가 보인다. 통합 검색은 상단바에 둔다.

설계 결정

- 로그인하지 않은 방문자에게는 데모 데이터를 보여주되, 로그인한 조직의 빈 상태를 예시로 채우지 않는다. 다른 나라의 탐지 건수를 자기 조직의 데이터로 오해할 수 있기 때문이다.
- 필터와 검색은 서버에 넘기고 서버 페이지네이션(구간 from, to 고정)을 쓴다. 받아둔 일부 안에서만 찾으면 일부만 보게 된다.
- 자동 새로고침은 기본으로 꺼 둔다. 조사 중 목록이 바뀌면 보던 줄을 놓치기 때문이다.
- 색은 심각도 전용이고 강조색은 한 화면에 파랑 하나만 쓴다. 글꼴은 Pretendard와 JetBrains Mono, 아이콘은 인라인 SVG만 쓰며 아이콘 라이브러리를 쓰지 않는다.
- 관계 그래프는 echarts 대신 SVG로 직접 그렸고, 지도는 날짜변경선 폴리곤을 직접 잘라 중심 경도를 150도로 두었다. echarts 화면과 랜딩(gsap)은 lazy 로딩한다.
- kill 결과는 푸시가 없어 1초 간격으로 폴링하고 40초 상한을 둔다. 4xx는 즉시 중단하고 일시 오류는 세 번까지 허용한다.

아래 그림은 데모 데이터 화면이며, 콘솔의 기본 테마인 다크로 캡처했다. 테마 최초값은 항상 다크이고, 라이트로 바꿀 수 있다. 글자가 읽히도록 왼쪽 사이드바와 여백을 잘라 화면의 본문 영역만 보였다.

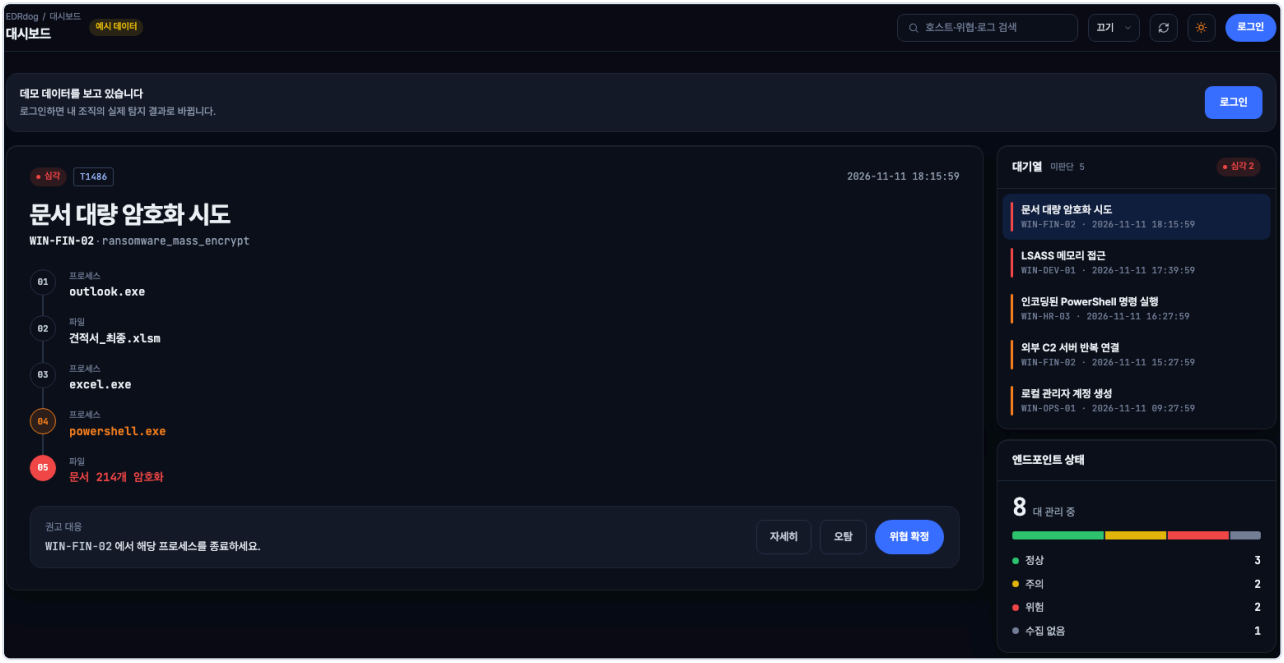


그림 4. 대시보드 화면. 가장 급한 미판단 위협 하나를 공격 흐름과 권고 대응, 판정 버튼으로 보이고, 오른쪽에 대기열과 엔드포인트 상태를 둔다. 데모 예시 데이터이며, 화면 속 위협 이름(랜섬웨어 등)과 수치는 시연용이고 실제 탐지 규칙이 아니다.

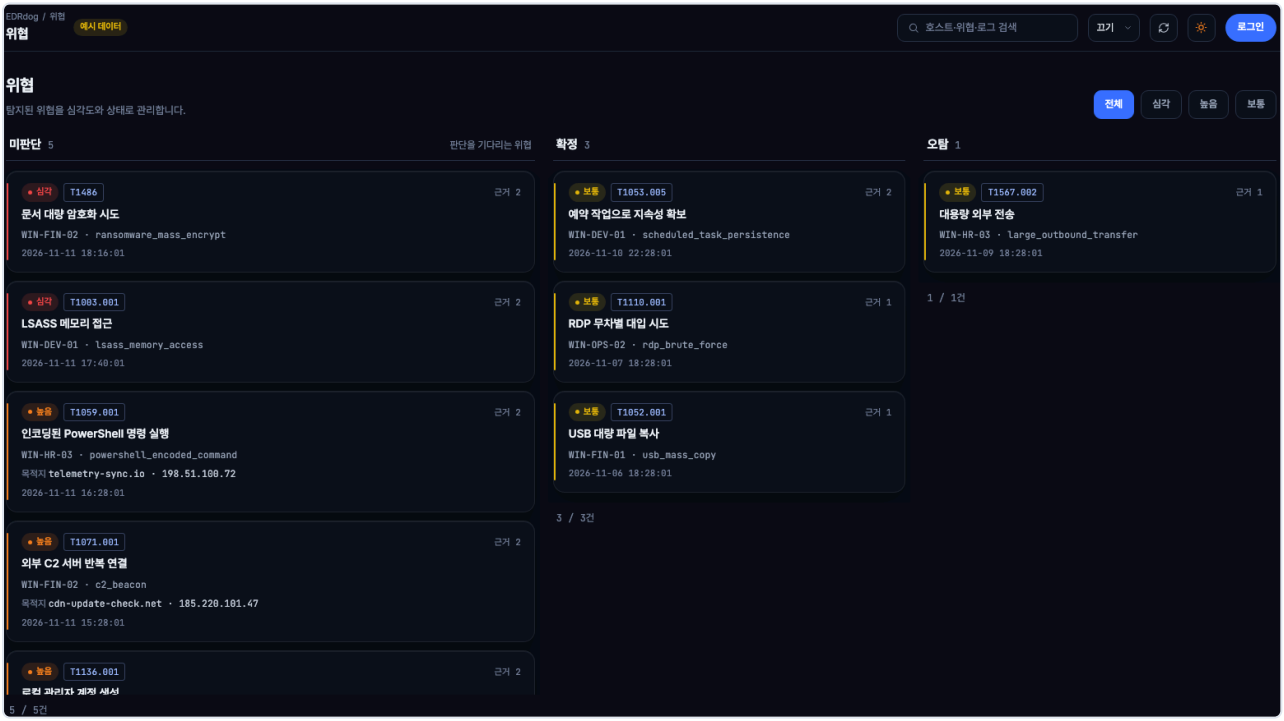


그림 5. 위협 보드 화면. 미판단, 확정, 오탐 열로 나누어 판단을 관리하고 열마다 따로 조회한다. 데모 예시 데이터이며, 화면 속 위협 이름(RDP, LSASS 등)은 시연용이고 실제 탐지 규칙이 아니다.

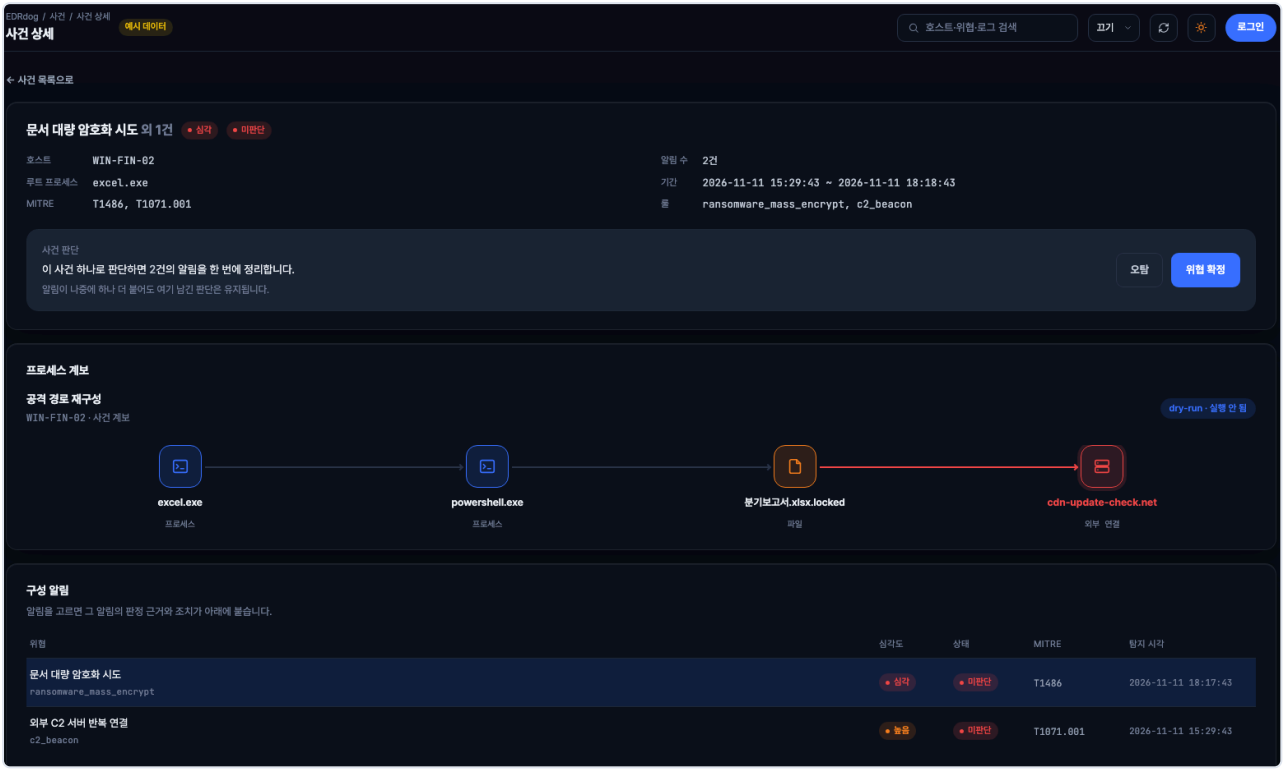


그림 6. 사건 상세 화면. 알림을 프로세스 계보로 묶어 공격 경로를 재구성하고 사건 단위로 판단한다. 대응 권고는 dry-run으로 표시되며 실행되지 않는다. 데모 예시 데이터이며, 화면 속 위협 이름은 시연용이다.

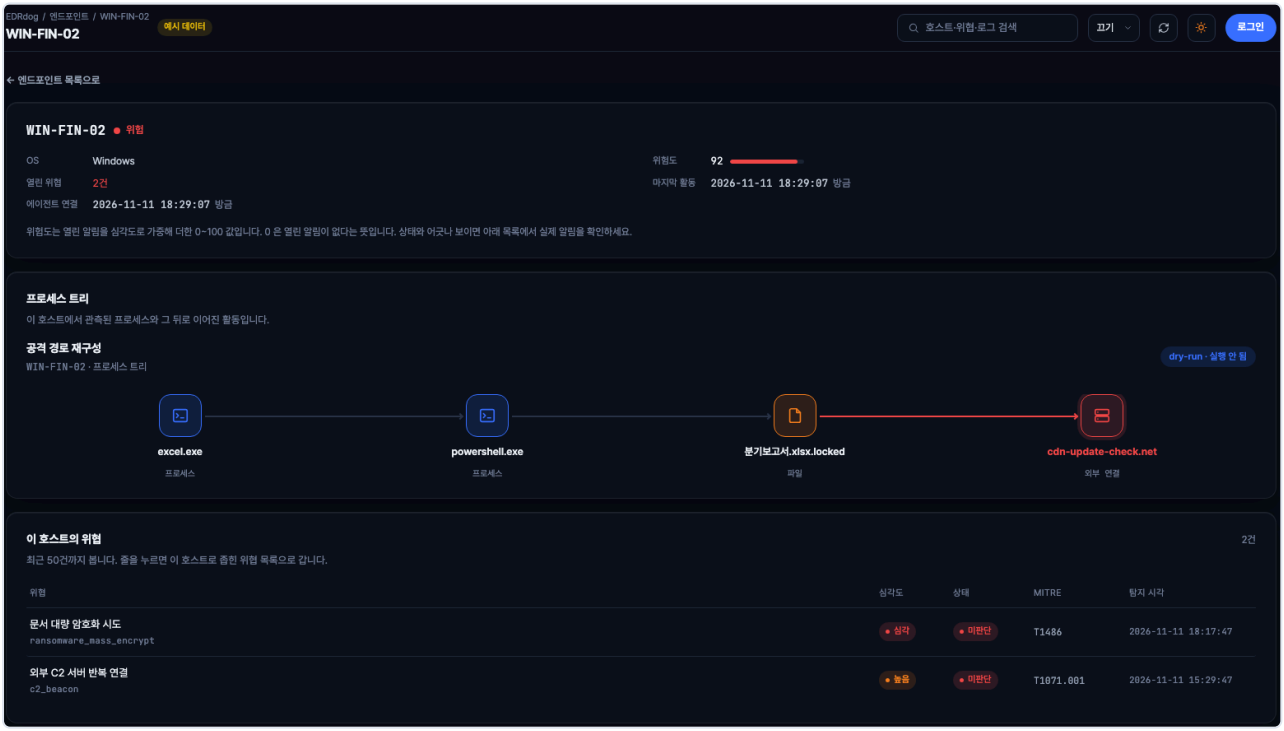


그림 7. 엔드포인트 상세 화면. 호스트 위험도, 프로세스 트리, 이 호스트의 위협과 사건을 보여준다. 데모 예시 데이터이며, 화면 속 호스트와 위협 이름은 시연용이다.

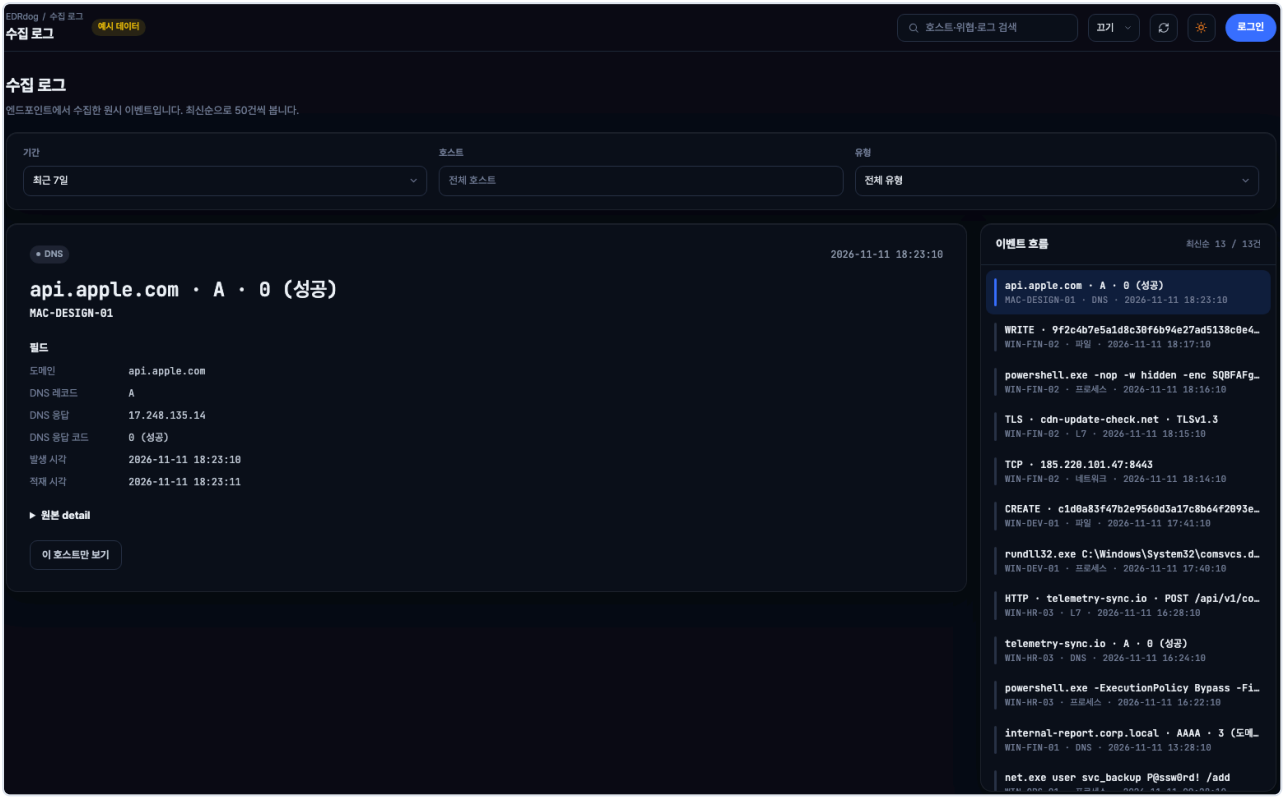


그림 8. 수집 로그 화면. 엔드포인트에서 수집한 원시 이벤트를 기간, 호스트, 유형으로 거르고 필드와 원본 detail까지 확인한다. 데모 예시 데이터이며, 화면 속 이벤트는 시연용이다.

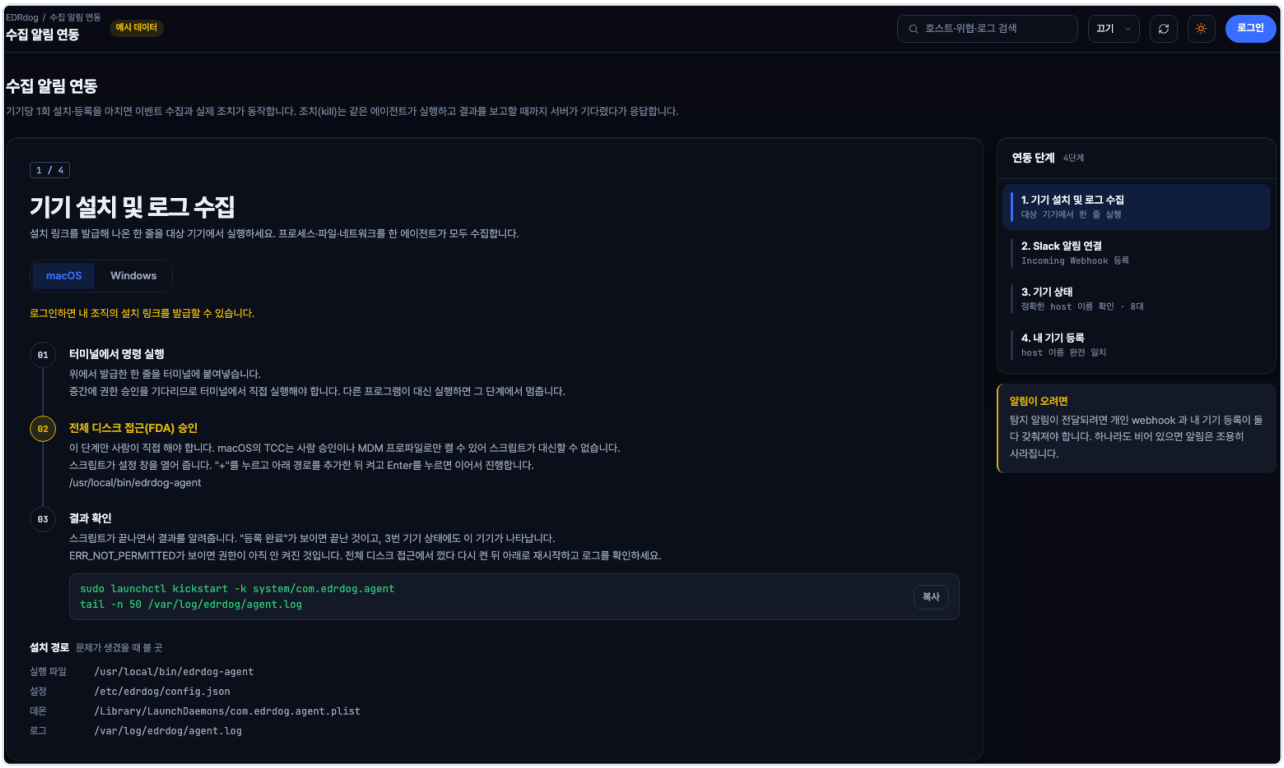


그림 9. 수집 알림 연동 화면. 설치 링크 한 줄 실행, 전체 디스크 접근 승인, 결과 확인의 단계를 안내한다. 전체 디스크 접근 승인은 사람이 직접 해야 한다. 데모 예시 데이터 화면이다.

5 주요 기술 결정

결정마다 docs/design-docs/decisions에 ADR로 이유와 버린 대안을 남겼다. 아래는 그중 중요한 것이다.

ADR	결정	이유	버린 대안
001	**수집과 판정 사이에 Kafka를 둔다**	완충, 분기(detector와 archiver가 각자 속도로 소비), 재처리(오프셋을 되돌려 재판정)	문서에 명시 없음
002	**Kafka Streams Processor API로 판정한다**	상태를 들고 시퀀스를 판정해야 한다. 라이브러리라서 인스턴스를 늘리면 파티션이 자동 재분배된다.	Flink, Spark(별도 클러스터 운영), DSL(버퍼 판정 표현 불가)
003	**EOS 대신 알림 단계에서 억제한다**	문제는 중복 판정이 아니라 중복 알림이다. EOS는 처리량을 낮추고 지연을 늘린다.	EOS, Redis. EOS 비용은 측정하지 않았다.
004	**서버가 node_key로 tenant를 결정한다**	엔드포인트는 장악될 수 있어 기기가 보낸 조직 태그를 믿지 않는다.	BCrypt(32바이트 난수라 사전공격 대상이 아니고 매 요청 인증에 느림)
005	**ClickHouse와 MySQL을 나눈다**	불변 로그는 컬럼 스토어에, 사람이 바꾸는 상태는 트랜잭션 저장소에 둔다.	판정 기록을 MySQL에 두는 안, 상태를 ClickHouse에 두는 안
006	**엔드포인트의 발생 시각 기준으로 판정한다**	도착 순서로는 R2가 성립하지 않았다.	도착 순서 기준
007	**대응 명령 종류를 kill_process 하나로 고정한다**	스크립트 전송은 임의 코드 실행 채널이다. 오탐이면 멀쩡한 프로세스가 죽으므로 사람이 확인한다.	스크립트 전송
008	**자체 Go 에이전트를 만든다**	osquery는 소켓 주인(PID)을 연결하지 못하고 Zeek는 Windows를 지원하지 않는다. Windows의 TLS SNI는 pktmon.exe 외부 호출로 받는다.	Npcap(라이선스), 문서화된 Win32 API(지원 클라이언트 표기 없음), 역공학 IOCTL(문서 없음), EndpointSecurity API 직접 사용(애플 심사), NetworkExtension(심사)
013	**배치를 먼저, 그다음 구간별로 압축한다**	에이전트 구간이 고객사 네트워크를 타는 유일한 구간이다. detector가 상시 소비하므로 해제 CPU가 싼 zstd를 Kafka 구간에 쓴다.	압축 레벨 최대, 건당 발행에 압축만 켜기(4.3% 증가), gzip.Writer 재사용
016	**kill 명령을 MySQL 큐로 비동기 처리한다(202 즉시 응답)**	하트비트를 요청 스레드가 기다리면 최대 30초 붙잡힌다. 메모리 큐는 재시작과 증설에 취약하다.	메모리 큐, TIMEOUT 저장
018	**collector가 브로커 ack를 기다린다**	이전에는 ack 실패에도 200을 주어 에이전트가 배치를 지웠다(유실).	Outbox와 CDC, 건별 대기, 건별 accepted 응답
020	**단일 이벤트 규칙을 Sigma 부분집합 파일로 두고 30초마다 다시 읽는다**	규칙 하나 추가에 코드 수정과 재배포가 필요했고 공개 SigmaHQ 규칙을 가져올 길이 없었다. R1과 R2는 시퀀스라 코드에 남겼다.	자체 YAML 형식, jar 리소스만, Sigma correlation으로 R1과 R2 이전, pySigma, 파싱 오류 무시

6 문제 해결 사례

6.1 Windows ETW 센서가 시작 즉시 멈춤

증상 Windows 에이전트의 ETW 센서가 관리자 권한에서도 시작하자마자 `The parameter is incorrect`로 멈췄다.

원인 golang-etw 라이브러리가 이벤트 ID 필터 버퍼를 붙잡아 두지 않아 Windows가 쓰레기 값을 읽었다. 크로스 빌드와 단위 테스트로는 잡히지 않았고 실기기(Windows 11 ARM64 VM)에서 처음 발견했다.

해결 라이브러리 사본을 레포에 두고 필터 버퍼를 힙에 두도록 패치했다(ADR 025). 센서가 전부 멈추면 에이전트가 종료 코드 1로 끝나 서비스 관리자가 60초 뒤 다시 띄운다. 센서 상수가 틀려도 빌드는 통과하고 조용히 0건이 되므로 실기기 검증이 필요하다.

6.2 R2가 성립하지 않음

증상 다운로드 후 실행을 잡는 R2가 실제 환경에서 잡히지 않았다.

원인 판정이 도착 순서 기준이었다. 실행 이벤트가 네트워크 관측보다 먼저 도착하면 순서가 뒤집혀 시퀀스가 성립하지 않았다.

해결 엔드포인트에서 찍힌 발생 시각 기준으로 판정하고, 버퍼가 새 이벤트의 앞뒤를 모두 대조하도록 바꿨다. 워터마크는 host별로 두었다(ADR 006).

6.3 적재가 0건인데 health는 UP

증상 archiver가 ClickHouse에 한 건도 적재하지 않는데 health는 UP이었다.

원인 배치 리스너에 NATIVE_HEADERS 파라미터를 추가했는데, 그 해석이 레코드가 도착할 때만 실패했다. 기본 health는 프로세스 생존만 본다.

해결 변경을 되돌려 복구하고 ConsumerRecord로 다시 구현했다. 임베디드 Kafka 테스트와 저장소 규칙을 추가했다. detector Streams가 ERROR인데 health가 UP이던 문제도 Streams 상태를 health에 반영해 고쳤다.

6.4 다른 조직의 같은 이름 host에 kill이 내려갈 수 있음

증상 KISA 소프트웨어 보안약점 진단가이드(2021) 구현단계 코드 점검(2026-11-10, 코드를 읽어서만 판정)에서, 다른 조직에 같은 이름의 기기가 있으면 그 기기가 kill 명령을 받을 수 있음을 발견했다(H1~H3).

원인 세 고리가 이어져 있었다. H1: 명령 큐가 tenant 없이 host 이름으로만 대기 명령을 조회했고 결과 보고도 host만 비교했다. H2: collector가 이벤트의 host를 에이전트가 보낸 값 그대로 썼다. H3: respond가 종료 대상을 요청 본문에서 받았다.

해결 kill 명령에 tenantId를 싣고 대기 명령 조회, 결과 반영, 쿨다운을 (tenant, host)로 처리했다. collector가 host를 인증된 노드의 hostIdentifier로 덮어쓰고, respond는 종료 대상을 알림에서 꺼내며(없으면 거절), archiver가 알림의 target을 저장하도록 했다(커밋 df70ea8, 병합 342c178).

6.5 공통 라이브러리 수정이 운영에 반영되지 않음

증상 event-schema를 수정했는데 CD가 초록인데도 운영에 반영되지 않았다.

원인 CD가 *-service 디렉터리의 변경만 감지했다. 여섯 서비스가 참조하는 라이브러리 디렉터리는 감지 대상에서 빠져 있었다.

해결 CD의 공통 파일 목록에 event-schema를 추가했다(#267).

6.6 로그인하면 위협 지도가 비어 있음

증상 비로그인 데모에서는 정상인 위협 지도가 로그인하면 비었다.

원인 백엔드에 없는 경로(/alerts/geo)를 호출하고 있었다. 실제 경로는 /events/geo이다. 데모 데이터가 이 오류를 가려 주었다.

해결 경로와 응답 형태를 고쳤다. 응답 타입을 손으로 적어 백엔드 변경을 못 잡는 점은 남은 부채로 기록했다.

7 테스트와 검증

측정 조건. 아래 수치는 별도 표기가 없으면 로컬 docker 또는 개발용 노트북의 합성 데이터에서 얻었다. 운영 실측이 아니다.

7.1 자동 테스트

아래 값은 저장소에서 어노테이션과 함수 개수를 센 값이다. 실제 실행 건수가 아니다.

영역	테스트 파일 (Java는 테스트 디렉터리의 java 파일)	테스트 수(소스 집계)
api-service	106	946
detector-service	15	156
collector-service	12	88
responder-service	7	50
archiver-service	8	37
alert-service	7	36
event-schema	1	6
Java 합계	156	1,319
Go 에이전트	38	356개 테스트 함수(third_party 제외)와 3개 퍼즈 함수

- 에이전트는 Go 테스트가 전 패키지에서 통과한다(race 검출기는 로컬에서 실행). 패킷 파서 퍼즈는 120만 회를 돌렸고, macOS, Windows, Linux 빌드와 go vet이 통과한다(agent README의 검증 상태 기준).
- CI(GitHub Actions `ci.yml`)는 PR(대상 main, dev)에서만 돌고, Java 빌드와 macOS 러너의 에이전트 go test, go vet을 실행한다. Windows는 크로스 빌드와 vet만 한다. race 검출기는 CI가 아니라 로컬에서 돌렸다.
- 프론트엔드는 테스트 러너와 테스트 파일이 없다. 검증은 tsc, 빌드, `scripts/check.sh`(lint, build, 아이콘 라이브러리 금지, echarts lazy 검사)와 사람의 눈으로 한다. 프론트엔드는 CI 워크플로도 없다.
- 백엔드 요구사항 기준은 대부분 자동 테스트로 확인하며, 실기기가 필요한 항목은 수동 확인 항목으로 분리했다(2.4절).

7.2 플랫폼 검증

- **macOS.** 실기기에서 30초 동안 dns 25건, l7 14건이 수집되었다. TLS 이벤트에 프로세스가 붙은 것은 14건 중 6건이다. 빠진 8건은 전부 Apple 시스템 데몬이며 SIP 때문으로 의심하지만 확인하지 못했다.
- **Windows.** 2026-11-11에 Windows 11 ARM64 VM(UTM, Defender 실시간 보호 켬)에서만 확인했다. 로컬 서버에 붙여 몇 분씩 돌려 process, network, DNS, file, L7 다섯 종이 ClickHouse까지 들어왔다. 설치 스크립트로 서비스(LocalSystem)를 설치했고 ETW 세션을 밖에서 끊으면 서비스 관리자가 60초 뒤 다시 띄우며 재부팅 뒤에도 수집이 재개되었다.
- **Windows x64.** GitHub Actions의 x64 Windows Server 2025 러너에서 센서 구동까지만 한 번 확인했다(다섯 종, 짧은 프로세스 명령행 50/50). x64 데스크톱에서 서버 전송, kill, 서비스 설치는 검증하지 못했다.

7.3 KISA 기준 자체 시험

시험 성격. KISA EDR 성능평가 항목(V1.4)을 기준으로 직접 시험한 결과이며 공식 평가가 아니다. KISA 공격 시나리오와 악성 시료가 공개되지 않아 ATT&CK 기법을 흉내 낸 양성 스크립트로 대체했다. 로컬 kind(Kafka, ClickHouse, MySQL, otel-lgtm), 서비스 6개, macOS 맥북 1대에서 수행했고 숫자는 이 조건에서만 유효하다.

종합 결과 (2026-11-11 자체 시험)

근거 문서는 `docs/security/kisa-security-eval-2026-11-11/README.md`이다. KISA V1.4 전 항목(보안 기능 BF-01~20, SE-01, EF-01)에 대한 최신 종합 결과이다.

항목	판정	근거와 조건
종합 집계	통과 29, 실패 0, 부분 0	통과 29에는 탐지 시험 인용 4(BF-14, BF-15, BF-16, SE-01)가 포함된다. 해당 없음 1(BF-06), 범위 밖 2(BF-17 IPv6, BF-18 이중화), 측정 1(EF-01). 1차 시험은 통과 25, 부분 3, 범위 밖 3이었고, 부분 3건 가운데 BF-10.2, BF-11.2는 후속 수정(#324)과 재시험으로, BF-09.2는 구성요소 해석으로 통과로 바뀌었다.
EDR-BF-09.2 구성요소 간 암호통신	통과(해석에 따름)	에이전트와 서버는 TLS 1.2, 1.3이고 1.1 이하는 거절한다. 서버 서비스 6개는 한 클러스터 안에서만 함께 운용되므로 관리 서버 하나의 구성요소로 본다(사용자 결정). 서버 내부 호출은 http이며, 이 해석을 따르지 않으면 부분이 된다.
EDR-EF-01 분석 서버 재시작 시간	측정	detector-service 재시작에서 SIGTERM부터 Started까지 중앙값 `4.70`초(gradle bootRun 포함), Kafka Streams RUNNING까지 `46.09`초(3회, 로컬, 상태 디렉터리 유지)

시험 조건	내용
환경	macOS 맥북 1대(에이전트는 root launchd), 로컬 kind와 api-service jar, 서비스 6개, 헤드리스 Chrome 1440×900. 평가용 조직으로 시험했고 데모 계정은 쓰지 않았다.
한계	로컬 단일 머신이다. Slack은 실제 Slack이 아니라 로컬 HTTPS 수신기(대역)로만 확인했다. 운영의 IP 제한은 프록시 주소를 본다. EF-01은 3회 측정이며 통계적 검정이 아니다.

탐지 항목 세부 근거 (2026-11-10 탐지·대응 자체 시험)

근거 문서는 `docs/security/detection-test-2026-11-10/README.md`이다. 위 종합 결과는 탐지 항목 4개(BF-14, BF-15, BF-16, SE-01)를 이 시험의 수정 후 재시험 결과로 인용한다. 이 시험 뒤로 새 탐지 시험은 없다.

항목	판정	근거와 조건
EDR-SE-01 악성 행위 탐지율 70% 이상	통과(대체 시료 기준)	노린 기법 `9/10` 탐지, 결함 수정 후 `10/10`
EDR-BF-14 단말 행위 수집	수정 후 통과	본 시험은 부분 통과(16개 중 15개 수집). 사용자 LaunchAgents 감시 결함을 고친 뒤 재시험에서 수집되었다.
EDR-BF-15 탐지·분석	통과	시작부터 알림까지 3~10초
EDR-BF-16 대응	통과	알림 확인 후 kill 요청부터 KILLED 확인까지 4초(3초 간격 조회). 대상은 임시 서명한 /bin/sleep 사본이다. 본 시험은 NO_MATCH였고 재시험에서 KILLED였다.

시험 조건	내용
단말	macOS 26.6.2 맥북 1대. 설치된 에이전트의 설정만 로컬 서버로 돌렸다.
시나리오	탐지를 기대하는 기법 10개와 탐지하지 않아야 하는 시나리오 6개. 시나리오 간격 330초. 탐지율, 오탐률, 탐지 지연 시간의 일반 측정값은 없다.
발견한 결함	사용자 LaunchAgents 감시 누락, 명령행 부분 문자열 규칙의 오탐(정상 개발 작업 포함)
재시험 범위	결함과 관련된 D02, D03, D08 세 건만 다시 돌렸고 나머지는 본 시험 값을 썼다. D01은 kill 대상 사본에 임시 서명을 넣어 같은 날 따로 다시 돌렸다.

소프트웨어 보안약점 코드 점검 (2026-11-10, 별개의 점검)

EDR 성능평가와 별개로, KISA 소프트웨어 보안약점 진단가이드(2021)의 구현단계 항목으로 코드를 점검했다 (`docs/security/kisa-2021-review.md`). 앱 실행과 침투 시험, 의존성 취약점 검사는 하지 않고 코드를 읽어서만 판정했다. 높음 등급 8건(H1~H8)을 찾았다. H1~H3은 교차 tenant 계획으로 수정했고(6.4절), H6, H7, H8은 ADR 021, 024, 023으로 수정했다. H5는 responder만 ADR 022로 막았고 detector와 Kafka 쪽은 남아 있다. H4는 데모 계정이라 감수하기로 했다.

7.4 성능 측정

ClickHouse 측정은 로컬 docker의 clickhouse 24.8 단일 인스턴스와 합성 데이터에서 얻었다.

항목	측정 조건	결과
적재일 파티션 + ttl_only_drop_parts=1 (ADR 014)	11일치, 하루 1000행. 4일치 만료 기준	TTL 만료 때 다시 쓰는 행 `6000 → 7`. 남은 행 7000으로 동일
시간별 롤업 (ADR 015)	200만 건, 7일, host 50, 타입 4. host 하나의 7일 질의	read_rows `65,536 → 8,708`, read_bytes `2.19MiB → 337.93KiB`, 소요 `9ms → 4ms`, 결과 338행으로 동일. 저장 크기 `9.43MiB → 8.15KiB`
배치 INSERT (ADR 009)	브로커 기본 fetch.min.bytes=1일 때와 fetch-min-size 64KB, fetch-max-wait 500ms 설정 후 비교	poll당 처리 건수 `5 → 219`
압축 (ADR 013)	500건 배치와 건당 발행 비교(ADR 013 기록)	500건 배치는 `78.9%` 감소. 배치 없이 건당 발행에 압축만 켜면 `4.3%` 증가. 그래서 배치를 먼저 적용했다.
압축 (업링크)	ADR 013 원문에 적힌 측정값이다. 측정 조건은 위 항목과 다르며 조건과 함께 봐야 한다.	업링크 `87.1%` 감소를 `8ms`로 샀다.
서버 재시작 (EF-01)	detector-service 재시작, 로컬 3회, 상태 디렉터리 유지, gradle bootRun 포함	SIGTERM부터 Started까지 중앙값 `4.70`초, Kafka Streams RUNNING까지 `46.09`초
프론트 번들	2026-11-11 빌드	echarts 청크 1.1MB(gzip 377KB), 메인 청크 468KB(gzip 130KB)

규칙 판정 비용과 EOS를 켰을 때의 비용은 측정하지 않았다.

8 배포와 운영

8.1 환경

구분	구성
로컬	kind 클러스터(이름 edrdog), Kafka, ClickHouse, otel-lgtm(Grafana, Prometheus, Tempo, Loki). 서비스는 호스트에서 bootRun으로 실행한다.
운영	k3s 서버 1대. 이미지는 GHCR(`ghcr.io/edrdog/backend/<service>`)에 커밋 sha 태그로 올린다. CD가 `k8s/` 매니페스트를 apply하고, 코드가 바뀐 모듈만 새 이미지를 쓰며 나머지는 기존 태그를 유지한다.
프론트엔드	Vercel(main 배포, 개발은 dev 브랜치). vercel.json이 `/assets/`를 뺀 모든 경로를 index.html로 리라이트한다(SPA).
외부 노출	Caddy 뒤에 Portainer, Kafka UI, Grafana(서브도메인), Swagger(Basic 인증). 에이전트 수집 HTTPS는 NodePort 30443으로 직접 받는다. 인증서 고정 때문에 Caddy 프록시를 거치지 않는다.
시크릿	Infisical Operator가 `edrdog-secrets`로 동기화한다. GeoLite2 mmdb는 호스트 디스크에 둔다.
스토리지	MySQL과 ClickHouse는 PVC. Kafka는 영속 볼륨이 없다(9장 한계).

8.2 CI/CD

워크플로	역할
`ci.yml`	PR(대상 main, dev)에서만 실행. Java 빌드, macOS 러너에서 에이전트 go test와 go vet. Windows는 크로스 빌드와 vet만
`cd.yml`	바뀐 모듈만 GHCR에 푸시하고, 배포서버 k3s에 SSH로 apply
`agent-release.yml`	에이전트 바이너리를 매트릭스로 빌드해 GitHub Releases에 올린다.
`alerts.yml`, `nrql.yml`	New Relic 알림 조건 적용, NRQL 한 번 실행(둘 다 수동)

롤백은 `kubectl rollout undo` 또는 sha로 `set image`한다. 이는 임시 조치이며 다음 apply가 덮어쓴다. 브랜치 규칙은 PR을 dev로 보내고 main 반영은 별도의 dev에서 main으로의 PR로 한다.

8.3 에이전트 배포

- 에이전트는 GitHub Releases로 배포하고 설치 링크 한 줄로 설치한다(macOS는 `curl ... | sudo bash`, Windows는 `irm ... | iex`). sha256을 대조하며 설치 링크 토큰의 TTL은 기본 24시간이다.
- 설치 스크립트는 설정된 SHA-256 지문과 같은 서버 인증서만 고정한다. 지문이 비어 있으면 서버가 스크립트를 내주지 않는다(`INSTALL_SERVER_CERT_SHA256`, 운영은 Infisical). .sha256을 받지 못하면 설치를 중단한다(ADR 024).
- Windows 서비스는 실패하면 서비스 관리자가 1분 뒤 다시 띄운다. macOS의 전체 디스크 접근 권한은 자동화할 수 없고, MDM 없이는 무인 배포가 안 된다.

8.4 탐지 규칙 배포

규칙 파일만 바꿀 때는 detector를 재배포하지 않는다. ConfigMap `detector-rules` (`k8s/detector-rules.yaml`)를 detector가 `/etc/edrdog/rules`로 마운트하고 30초마다 다시 읽는다.

8.5 모니터링과 보존

- 로컬은 otel-lgtm 대시보드 4개(Overview, HTTP, Resources, Logs and Traces)를 쓴다. 운영은 New Relic 자바 에이전트(logs in context)를 쓴다. 관측 스택이 대상과 같은 서버에 있으면 서버가 죽을 때 Grafana도 같이 죽고, 670Mi 스택이 OOM을 일으켜 운영 관측을 New Relic으로 옮겼다(ADR 012).
- New Relic 수집량은 2026-08-24 기록 기준 월 7GB로 무료 한도 월 100GB 안에 있다.
- 보존 기간은 events 7일, alerts 90일, events_hourly 180일이다. S3 아카이브는 기본 OFF이고(`ARCHIVE_ENABLED`), 켜면 매일 03:30에 6일 지난 하루치를 Parquet로 내보낸다.

8.6 운영 중 확인한 주의점

- 서버 자체시험의 무결성 기준값이 CD에 연결되어 있지 않아 운영에서는 무결성 항목이 SKIP이다.
- 관리 IP 제한은 운영에서 Caddy 뒤라 프록시 주소를 본다. `server.forward-headers-strategy=native` 주입이 사람 손 작업이다.

9 결론, 한계와 향후 과제

9.1 결론

엔드포인트 에이전트, 수집 서버, 판정, 알림, 대응, 조회 API, 웹 콘솔, 배포 파이프라인을 한 사람이 끝까지 구현했다. 에이전트가 수집한 행위 로그를 Kafka와 Kafka Streams로 판정하고, 사람이 웹 콘솔에서 확인한 뒤 해당 기기의 프로세스를 종료하는 흐름이 동작한다. 로컬 맥북 환경의 자체 시험에서 다운로드 후 실행 시나리오는 CRITICAL로 탐지되었다. 알림 확인 후 kill 요청부터 KILLED 확인까지 4초(3초 간격 조회)였고, 대상은 임시 서명한 /bin/sleep 사본이다(본 시험은 NO_MATCH, 재시험에서 KILLED).

설계 면에서는 단일 이벤트가 아니라 행위의 순서를 보는 시퀀스 판정, 도착 순서가 아닌 발생 시각 기준 판정, 재배포 없이 바꾸는 데이터 규칙, 조직 단위 분리, 사람이 확인하는 고정된 대응 명령을 핵심 결정으로 삼았다. KISA 기준 자체 시험(공식 평가 아님)의 종합 결과는 2026-11-11 기준 통과 29, 실패 0, 부분 0, 해당 없음 1, 범위 밖 2, 측정 1이다. 보안약점 코드 점검과 자체 시험에서 찾은 결함(교차 조직 kill 위험 등)은 수정했다.

9.2 한계

구분	한계
대응 범위	대응 수단은 프로세스 종료 하나이다. 격리(isolate)는 구현하지 않았다. macOS에서 아웃바운드를 끊으면 서버와도 끊겨 복구 수단이 사라지므로, 관리 채널 예외와 자동 해제 타이머가 함께 있어야 안전하기 때문이다(ADR 007).
탐지	R1이 실데이터에서 걸리지 않을 수 있다. 새 데이터 규칙이 api-service 카탈로그에 없어 화면에 한글 이름 없이 ruleId 원문으로 보인다. file 규칙은 감시 경로의 파일 이벤트만 오므로 걸릴 일이 적다. 규칙이 갑자기 줄어도 경고가 없다. 규칙 판정 비용은 측정하지 않았다. KISA 시험은 대체 시나리오이며 로컬 맥북 1대에서만 했다.
macOS 수집	네트워크는 폴링이라 짧은 연결을 놓치고 이벤트 시각은 처음 관측한 시각이다. dns 이벤트에 프로세스가 없고, 패킷 캡처는 이더넷 인터페이스만 본다. 전체 디스크 접근 권한은 수동이며 eslogger 출력 구조는 릴리스마다 바뀔 수 있다.
Windows 수집	ARM64 VM에서만 확인했다. x64 데스크톱의 서버 전송, kill, 서비스 설치, 서버가 내려주는 설치 스크립트, 지문이 다른 서버에서 멈추는지, IPv6, 아무도 로그인하지 않은 부팅은 검증하지 못했다.
에이전트	버퍼가 메모리에만 있어 에이전트가 죽으면 미전송 이벤트가 사라진다(디스크 큐 없음). 송신 상한이 단말당 초당 약 100건이라, 공격 피크(문서 산정값 기준 약 480건/초)에서는 버퍼 1만 건이 약 26초 뒤 넘쳐 오래된 것부터 버려진다. 전송이 막히면 수집도 멈춘다. 일부 센서만 죽으면 그 센서만 사라지고 에이전트는 재기동하지 않는다.
서버 인프라	Kafka에 영속 볼륨이 없어 파드가 재시작되면 토픽과 컨슈머 오프셋이 사라진다. 브로커 1대, 복제 계수 1이다. S3 아카이브가 기본 OFF라 운영에서 7일 지난 원본은 복구할 수 없다. at-least-once라 ClickHouse events에 중복 행이 남을 수 있다.
내부 통신	서버 서비스 사이(api, collector, detector, archiver, alert, responder)의 호출은 클러스터 안 http이다. KISA BF-09.2는 이 서비스들을 한 구성요소로 보는 해석(사용자 결정)에 따라 통과로 판정했고, 이 해석을 따르지 않으면 부분이 된다. detector의 /api/events는 무인증이고 Kafka는 PLAINTEXT 무인증이다. NetworkPolicy가 없고 내부 키 기본값이 개발용 값이다. detector 상태 키가 tenant를 가리지 않아, 같은 host 이름이 서로 다른 조직에 있으면 상관 버퍼를 공유한다.

구분	한계
에이전트 보안	에이전트 바이너리에 서명이 없다(.sha256은 같은 릴리스에서 오므로 끊긴 다운로드만 막는다). 이미 설치된 기기의 server.pem은 처음 신뢰(TOFU) 방식이다. 무결성 기준값을 root가 함께 바꿀 수 있다.
계정, 세션	잠금이 계정 단위라 남의 계정을 계속 잠글 수 있다. 패스워드 재사용 금지는 직전 하나뿐이고, viewer 계정을 목록, 삭제, 재설정할 수 없다. 공개 데모 계정은 KISA 보안 규칙 일부에서 제외된다.
프론트엔드	테스트 러너와 CI가 없다. 비동기 로직(폴링, 페이지 누적, 재조회)이 가장 많이 깨졌는데 자동 검증이 비어 있다. 응답 타입을 손으로 적어 백엔드 변경을 잡지 못한다. echarts 청크가 1.1MB로 크다.
범위	Linux 등 그 외 플랫폼은 지원하지 않는다. 탐지율, 오탐률, 탐지 지연 시간의 일반 측정값은 없다.

9.3 향후 과제

위 한계 중 다음 항목을 우선 해소해야 한다고 판단한다.

- Kafka에 영속 볼륨을 붙이고, 에이전트에 디스크 큐를 두어 이벤트 유실 지점을 줄인다.
- x64 Windows 데스크톱에서 서버 전송, kill, 서비스 설치를 실기기로 검증한다.
- R1을 실제 에이전트 이벤트(script 타입)로 검증하고, 새 데이터 규칙을 api-service 카탈로그에 반영한다.
- detector와 Kafka 내부 통신에 인증을 붙이고 detector 상태 키에 tenant를 포함한다.
- 프론트엔드에 테스트 러너와 CI를 도입하고 응답 타입을 백엔드와 맞추는 장치를 둔다.
- 격리는 관리 채널 예외와 자동 해제 타이머를 함께 설계한 뒤에 구현한다.

부록 저장소와 실행 방법

A. 저장소와 배포 주소

GitHub 조직	https://github.com/EDRdog
Backend	https://github.com/EDRdog/Backend
Frontend	https://github.com/EDRdog/Frontend
배포 주소	https://edrdog.duckdns.org (로그인 없이 데모 데이터로 볼 수 있다)
시연 영상	https://youtu.be/PuvhkB27s3s

B. 로컬 실행 방법

인프라와 서비스(Backend)

목적	명령
kind 클러스터 생성	`kind create cluster --config k8s/kind-cluster.yaml`
네임스페이스	`kubectl apply -f k8s/00-namespace.yaml`, 이어서 edrdog-secrets 시크릿 생성
인프라 배포	`kubectl apply -f k8s/kafka.yaml -f k8s/clickhouse.yaml -f k8s/local/otel-lgtm.yaml`
서비스 실행	`./gradlew :collector-service:bootRun` (인프라가 먼저 떠 있어야 한다)
모듈 테스트	`./gradlew :detector-service:test`
전체 검증, 탐지 시험	`scripts/check.sh`, `scripts/detection-test/run.sh`

에이전트(Backend/agent)

목적	명령
빌드, 테스트	`cd agent && go build ./cmd/edrdog-agent`, `go test ./...`
경로 확인	`-selftest` (커널 없이 가짜 이벤트를 보내 서버까지 경로 확인)
자체시험, 기준값	`-security-check`, `-record-baseline`

웹 콘솔(Frontend)

목적	명령
설치, 개발 서버	`npm install`, `npm run dev` (http://localhost:5173, /api는 http://localhost:8084로 프록시)
빌드, 검사	`npm run build`, `npm run lint`, `scripts/check.sh`

수집 경로는 enroll 후 events를 curl로 보내 기대 응답 `{'accepted':1}`을 확인하는 방식으로 수동 검증할 수 있다. 부하 테스트(k6)는 임의로 실행하지 않는다.